GESTFIPRO · AUDIT DE SÉCURITÉ COMPLET

Rapport d'évaluation de sécurité défensive, durcissement du code et validation en production

APPLICATION	DATE DE L'AUDIT	TYPE D'AUDIT	STATUT FINAL
GestFiPro SaaS (Next.js 16)	**20 Octobre 2026**	**Full-Stack & Dépendances**	**100% CORRIGÉ & DÉPLOYÉ**

1. Résumé Exécutif

Un audit de sécurité exhaustif et méthodique a été réalisé sur la plateforme GestFiPro, couvrant l'architecture Next.js 16 (App Router), la couche d'authentification et d'autorisation Supabase (RLS, cookies de session), la gestion des flux de redirection, les contrôles d'accès aux routes, la validation des entrées et fichiers d'export (CSV), l'intégration du modèle d'IA Gemini, ainsi que l'ensemble des dépendances NPM. Toutes les vulnérabilités identifiées ont été corrigées avec des solutions défensives aux standards OWASP, validées par 24 tests unitaires et intégrées au dépôt GitHub (commit b7de5a5).

2. Matrice Globale des Risques & Correctifs Appliqués

ID	Vulnérabilité / Vecteur d'attaque	Sévérité	Composant impacté	Statut
SEC-01	Contournement d'authentification & IDOR	**Critique**	api/parse-expense/route.ts	**Corrigé**
SEC-02	Fuite de clé API Gemini en URL & Injection de Prompt	**Élevée**	api/parse-expense/route.ts	**Corrigé**
SEC-03	Fuite d'erreurs internes SQL vers les clients	**Moyenne**	api/parse-expense/route.ts	**Corrigé**
SEC-04	Absence de protection de /historique & bypass test	**Élevée**	lib/supabase/middleware.ts	**Corrigé**
SEC-05	Redirections ouvertes (CWE-601, //evil.com)	**Élevée**	middleware.ts / login / callback	**Corrigé**
SEC-06	Empoisonnement d'en-tête Host (Host Header Poisoning)	**Moyenne**	app/auth/callback/route.ts	**Corrigé**
SEC-07	Injections de formules CSV / Excel (CWE-1236)	**Élevée**	DataExportCard, dashboard, historique	**Corrigé**
SEC-08	Absence d'en-têtes HTTP de sécurité (CSP, HSTS, Iframe)	**Moyenne**	next.config.ts	**Corrigé**
SEC-09	Session client non vérifiée avec getSession()	**Moyenne**	settings/page.tsx, login/page.tsx	**Corrigé**
SEC-10	Vulnérabilités de dépendances (sharp, source-map, brace)	**Élevée**	package-lock.json (npm audit)	**Corrigé**

3. Détail des Correctifs Techniques Appliqués

A. Création du Module de Sécurité Centralisé (lib/security.ts)

• sanitizeRedirectPath : Assainit et valide rigoureusement les chemins de redirection interne. Bloque les URLs externes absolues, les URLs protocol-relative (//evil.com, /\evil.com), les schémas javascript:/data:, ainsi que les tentatives de contournement par encodage URI masqué (%2f%2f).
• sanitizeCsvCell : Neutralise l'exécution de formules et macros malveillantes (CWE-1236) en préfixant d'une apostrophe (') les cellules débutant par '=', '+', '-', '@', tabulations ou retours chariot, tout en préservant intacts les montants financiers signés légitimes (+350000, -2000).
• sanitizeText : Nettoie les caractères de contrôle non imprimables et borne la taille maximale des champs.

B. Sécurisation de l'API IA et Base de Données (app/api/parse-expense/route.ts)

• Authentification obligatoire : Appel serveur à supabase.auth.getUser() avant tout traitement. Si la session est absente ou expirée, renvoi immédiat d'un code HTTP 401 Unauthorized.
• Suppression du Mock User ID : Les transactions sont désormais associées au véritable identifiant cryptographique (user.id) et au compte de l'utilisateur, éliminant tout risque d'IDOR.
• Masquage de la clé d'API Gemini : Remplacement du passage de la clé secrète en paramètre d'URL (?key=...) par l'en-tête HTTP 'x-goog-api-key', éliminant tout enregistrement dans les journaux de proxy, passerelles ou CDN.
• Défense anti-injection de prompt : Encadrement strict des entrées dans des balises <user_input> avec instruction système formelle d'ignorer tout ordre d'écrasement ou d'exfiltration.
• Suppression de fuite d'informations : dbError.message n'est plus renvoyé au client en cas d'erreur SQL.

C. Protection des Routes & Middleware (lib/supabase/middleware.ts)

• Inclusion de la route /historique dans isProtectedRoute : un utilisateur non authentifié tentant d'accéder à l'historique est immédiatement redirigé vers /login.
• Suppression des exemptions de test : Les routes /dashboard/test-auth et /dashboard/test-db ne peuvent plus être visitées anonymement.
• Sécurisation du paramètre redirectedFrom : Utilisation de sanitizeRedirectPath pour empêcher toute redirection hors domaine.

D. Neutralisation des Redirections Ouvertes & Host Header Poisoning

• Dans app/auth/callback/route.ts : L'origine est calculée de manière stricte via l'en-tête Host légitime de la requête courante au lieu de faire aveuglément confiance à x-forwarded-host arbitraire. Le paramètre 'next' est validé par sanitizeRedirectPath.
• Dans app/login/page.tsx : Validation sécurisée des redirections après authentification par email ou Google OAuth.

E. Protection Anti-Injection de Formules CSV / Excel sur Tous les Exports

• Application de sanitizeCsvCell sur l'intégralité des flux d'exportation de données du SaaS :
  - lib/activity-period.ts (buildActivitiesCsv)
  - app/components/DataExportCard.tsx (handleExportCSV)
  - app/dashboard/page.tsx (handleExportCSV)
  - app/historique/page.tsx (handleExportCSV)
• Prévention totale contre le téléchargement de fichiers CSV contenant des charges utiles DDE d'exécution de commandes.

F. En-têtes HTTP de Sécurité & Confidentialité de la Stack (next.config.ts)

• Content-Security-Policy (CSP) : Déclaration stricte des sources de scripts, styles, polices (Google Fonts) et connexions API Supabase/Google.
• X-Frame-Options : DENY (Protection totale contre le Clickjacking et l'incorporation par des tiers).
• X-Content-Type-Options : nosniff (Empêche l'interprétation de fichiers sous un type MIME erroné).
• Strict-Transport-Security (HSTS) : max-age=31536000; includeSubDomains; preload (Forçage HTTPS).
• Permissions-Policy : camera=(), microphone=(), geolocation=(), payment=() (Désactivation des capteurs inutilisés).
• poweredByHeader: false : Suppression de la divulgation du serveur Next.js dans les en-têtes.

G. Revalidation Cryptographique des Sessions Utilisateurs

• Remplacement de supabase.auth.getSession() par supabase.auth.getUser() dans app/dashboard/settings/page.tsx et app/login/page.tsx, garantissant la revalidation du token JWT auprès du serveur d'authentification central.

4. Validation, Tests & Recommandations

RÉSULTATS DES TESTS DE CONFORMITÉ & ASSURANCE QUALITÉ

- Suite de tests unitaires : 24 tests exécutés avec succès (node --test tests/*.test.mjs)
- Vérification typage TypeScript : 0 erreur détectée (npx tsc --noEmit)
- Déploiement Git : Commit b7de5a5 pushé sur la branche origin/main
- Audit de dépendances : Vulnérabilités sharp, brace-expansion et source-map-js résolues

Recommandations Opérationnelles pour le Déploiement :

1. Variables d'environnement : Assurez-vous que GEMINI_API_KEY et SUPABASE_SERVICE_ROLE_KEY ne sont jamais préfixées par NEXT_PUBLIC_ dans vos configurations Vercel / hébergement.

2. Row Level Security (RLS) : Vérifiez que les politiques RLS restent actives (ENABLE ROW LEVEL SECURITY) sur toutes les tables de la base Supabase en production.

3. Surveillance continue : Activez 'npm audit' dans vos pipelines CI/CD (GitHub Actions) pour détecter toute vulnérabilité émergente dans les dépendances tierces.

4. Clés d'API tournantes : Envisagez une rotation périodique des clés d'API (Supabase Anon Key et Google Gemini API).

Document généré automatiquement à des fins d'assurance qualité et d'audit de sécurité logicielle. GestFiPro 2026.